

Rakhshan Yousaf, N1230793

Course: COMP30271: Cognitive Computing

Submission of Portfolio 1: Bayesian Programming for Perception

Submission of Portfolio 2: Navigation using ROS2 in Gazebo simulation

Submission of Portfolio 3: Probabilistic 3D Mapping, Localisation and Navigation

Table of Contents

1. Bayesian Programming for Perception.....	4
1.1 Introduction.....	4
1.2: Background.....	5
1.2.1 Sensory Integration in Human Perception.....	5
1.2.2 The Ventriloquist Effect and Sensory Capture.....	5
1.2.3 The Generative Model	5
1.2.4 The Ideal Bayesian Observer	6
1.3 System Design	7
1.3.1 Architecture Overview	7
1.3.2 Implementation Details.....	7
1.3.3 Performance Metrics	8
1.3.4 UML Diagrams	8
1.4 My Results.....	9
1.4.1 Test 1: Qualitative Analysis (10 Trials).....	9
1.4.2 Test 2: Quantitative Analysis (1000 Trials)	10
1.4.3 Statistical Power and Interpretation	11
1.5 My Conclusion - Summarising of what I've achieved	12
2. Navigation with ROS2 in Gazebo Simulation.....	13
2.1 Introduction.....	13
2.2 Background.....	14
2.2.1 Overview of Navigation Strategies	14
2.2.2 Collision Avoidance Approaches	14
2.2.3 ROS2 NAV2: Architecture and Components	15
2.3 System Design	16
2.3.1 Architecture Overview	16
2.3.2 Key ROS2 Packages Used	17
2.3.3 Key Nodes, Topics and Services	17
2.3.4 Simulation Environment Setup.....	18
2.3.5 UML Component Diagram (.....	18
2.3.6 Obstacle Detection and Path Planning.....	19
2.4 Results	19
2.4.1 Test Scenarios	19
2.4.2 Planner and Controller Comparison Results	20
2.4.3 Speed-Safety Trade-off Analysis (Hospital Robot Challenge)	21
2.4.4 Hospital Service Robot Recommendation	22
2.4.5 Collision Avoidance Effectiveness	23
2.4.6 Limitations and Potential Improvements	24
2.5 Conclusion of completed work	24
Part 3: Navigation with ROS2 in Gazebo Simulation	26
3.1 Introduction.....	26
3.2 Background.....	26
3.2.1 SLAM Strategies	26
3.2.2 Probabilistic Mapping with OctoMap	26
3.3 System Design	27
3.3.1 High-Level Architecture.....	27

3.3.2 Key ROS2 Packages and Algorithms.....	27
3.3.3 Nodes, Topics, and Services	27
3.3.4 Simulation Environment.....	28
3.3.5 Sensing and Obstacle Detection	28
3.4 Results	28
3.4.1 Work Completed	28
3.4.2 Problems Encountered.....	29
3.4.3 Analysis.....	29
3.4.4 Lessons Learned.....	29
3.5 Conclusion	30
References – accessed from jan 19th 2026 – 20th feb 2026	30

1. Bayesian Programming for Perception

1.1 Introduction

Human perception is an active inferential process whereby the brain estimates the most likely state of the external world by using noisy, ambiguous sensory inputs (Helmholtz, 1867; Colombo and Seriès, 2012). Hermann von Helmholtz articulated this as "unconscious inference"—automatic probabilistic computations combining sensory evidence with learned expectations about environmental regularities (Patton, 2021). Contemporary neuroscience has gathered this within a Bayesian framework, demonstrating that perceptual systems approximate statistically optimal inference across modalities (Kersten, Mamassian and Yuille, 2004; Parr, Rees and Friston, 2018).

Multisensory integration exemplifies this inferential machinery. When visual and auditory signals arrive near enough together or together, the brain faces a causal inference problem: so do these signals originate from a common source requiring integration, or independent events requiring separate processing? (Körding et al., 2007). Körding demonstrated that human audiovisual localisation behaviour is predicted by an ideal Bayesian observer model estimating the probability of common versus independent causes and the spatial locations of sources. This indicates that causal inference operates automatically in low-level perception.

The ventriloquist effect demonstrates sensory capture: the perceived location of an auditory stimulus shifts toward a spatially disparate but temporally coincident visual stimulus (Alais and Burr, 2004). This arises from optimal uncertainty handling—when evidence favours a common cause, the more reliable modality dominates the percept (Battaglia, Jacobs and Aslin, 2003).

Objectives:

This portfolio implements and evaluates a computational model of the ventriloquist effect to demonstrate a few Bayesian perception principles such as these below:

- Implement a generative model simulating audiovisual localisation trials, sampling causal structure, source locations, and noisy observations using Python statistical distributions.
- Apply an ideal Bayesian observer to perform causal inference and location estimation from generated data, replicating human-like ventriloquist biases
- Evaluate performance across trial counts (10 for qualitative analysis, 1000 for statistical robustness) using confusion matrices, mean absolute error, and sensory shift metrics.
- Interpret results within optimal multisensory integration theory, validating the Bayesian causal inference framework.

1.2: Background

1.2.1 Sensory Integration in Human Perception

The central nervous system continuously receives information from multiple sensory channels vision, audition, touch; each corrupted by independent noise processes (Battaglia, Jacobs and Aslin, 2003; Parr, Rees and Friston, 2018). Optimal perception requires combining these redundant signals to form unified, precise estimates of environmental properties. Maximum likelihood estimation formalises this integration problem: when estimating a single underlying variable from two noisy observations, the optimal combined estimate is a weighted average where weights are inversely proportional to sensory variances (Alais and Burr, 2004; Landy, Banks and Knill, 2011). This weighting scheme minimises the variance of the combined estimate, yielding precision superior to either uni-sensory signal alone. Psychophysical experiments across modality pairs confirm that adult humans integrate redundant cues in a manner statistically indistinguishable from these predictions (Battaglia, Jacobs and Aslin, 2003; van Beers, Sittig and van der Gon, 1999).

but forced-fusion models that assume a single underlying cause fail when sensory signals originate from independent sources (Körding et al., 2007; Beierholm et al., 2008). Integrating a nearby visual object with a distant auditory sound would yield a spurious unified percept. The brain must therefore solve a more complex problem: inferring causal structure before deciding how to combine sensory estimates (Körding et al., 2007; Colombo and Seriès, 2012).

1.2.2 The Ventriloquist Effect and Sensory Capture

It's great that sensory capture describes the phenomenon whereby one sensory modality dominates perception when cues conflict, pulling the percept toward the more reliable modality (Alais and Burr, 2004; Bruns and Röder, 2019). This occurs across numerous pairings: vision captures audition in spatial localisation, audition captures vision in temporal judgments, and touch captures vision in texture perception. The effect is not perceptual error but an optimal inference strategy given differential reliability - when visual uncertainty is much smaller than auditory uncertainty, visual information is weighted more heavily, causing the perceived auditory location to shift toward the visual stimulus (Alais and Burr, 2004; Battaglia, Jacobs and Aslin, 2003).

The ventriloquist effect specifically refers to audiovisual spatial capture (Bertelson and Radeau, 1981; Alais and Burr, 2004). When visual and auditory stimuli are presented simultaneously with spatial disparity, the perceived auditory location is biased toward the visual source. Critically, this bias decreases nonlinearly with increasing disparity: small disparities produce strong capture, while large disparities yield minimal bias as the brain infers independent causes (Körding et al., 2007; Alais and Burr, 2004). Bruns and Röder (2019) documented that the effect persists even with invisible visual stimuli under continuous flash suppression, indicating both bottom-up and top-down contributions. The ventriloquist aftereffect represents a distinct phenomenon where prolonged exposure to spatially discrepant audiovisual pairings recalibrates subsequent uni-sensory auditory localisation, suggesting separate mechanisms for immediate sensory capture and longer-term spatial recalibration (Bosen et al., 2016; Bruns and Röder, 2019).

1.2.3 The Generative Model

A generative model specifies the causal process by which hidden variables in the world produce observable sensory data (Kersten, Mamassian and Yuille, 2004; Parr, Rees and Friston, 2018). In Bayesian perceptual modelling, this encodes the experimenter's assumptions about environmental statistics and sensory noise characteristics. For the audiovisual localisation task, Körding et al. (2007)

defined a hierarchical structure where a Bernoulli-distributed causal variable indicates whether one common source or two independent sources generated the sensory signals. Under the common cause hypothesis, a single spatial location is drawn from a Gaussian prior and assigned to both modalities, enforcing spatial coincidence.

Under the independent source's hypothesis, separate locations are drawn independently from the same prior. Noisy sensory observations are then generated by adding Gaussian noise to these true locations, with visual and auditory noise standard deviations reflecting the differential reliability of each modality. This generative model defines a joint probability distribution that factorises according to the causal graph, enabling representation of both integration and segregation within a unified framework (Körding et al., 2007; Beierholm et al., 2008). This is what I had done similar in my `environment_setup.py` file.

1.2.4 The Ideal Bayesian Observer

An ideal observer performs optimal inference given the generative model by computing the posterior distribution over hidden variables conditioned on sensory evidence, then selecting estimates that minimise expected loss (Kersten, Mamassian and Yuille, 2004; Landy, Banks and Knill, 2011). The observer first computes the posterior probability of a common cause using Bayes' rule, marginalising over possible source locations. The likelihoods under each causal hypothesis are obtained by integrating over the unknown true locations, with closed-form Gaussian solutions available for this particular model structure. Given the posterior over causal structure, the optimal location estimates under mean squared error loss are weighted combinations of the estimates under each hypothesis, where the weights correspond to the probabilities of the causal structures. Under the independent sources hypothesis, the optimal estimates are simply the uni-sensory observations. Under the common cause hypothesis, the estimates are inverse-variance weighted combinations representing maximum likelihood fusion. Because the posterior probability of a common cause depends nonlinearly on the sensory inputs, the overall estimates are nonlinear mixtures rather than simple linear weighted averages. This nonlinearity explains partial integration: when evidence for a common cause is ambiguous, the observer hedges by weighting both hypotheses, producing intermediate biases (Körding et al., 2007; Beierholm et al., 2008).

Fitting this model to human data, Körding et al. (2007) reported parameter estimates of approximately 2.14 degrees visual noise, 9.2 degrees auditory noise, 12.3 degrees prior standard deviation, and 0.28 prior probability of common cause. The model explained most of the variance in bimodal response distributions and unity judgments, outperforming simpler forced-fusion models (Körding et al., 2007; Beierholm et al., 2008).

1.3 System Design

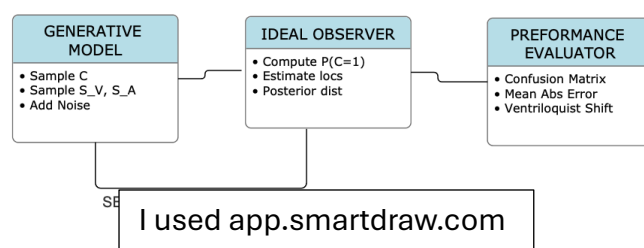
1.3.1 Architecture Overview

The implemented system comprises three modular components organised in a unidirectional data flow pipeline. The generative model simulates experimental trials by sampling from the probabilistic generative model described in Section 1.2.3, outputting arrays representing the true causal structure, true source locations, and noisy sensory observations. These feed into the ideal observer, which computes posterior probabilities and location estimates from sensory inputs according to Bayesian principles. Finally, the performance evaluation component calculates confusion matrices, mean absolute error, and bias metrics, producing visualisations for both small and large trial counts. Conceptually, the pipeline flows from generative model to sensory observations, then to Bayesian inference, and finally to performance metrics.

The generative model constitutes the primary implementation task for this portfolio. This module accepts experimental parameters including trial count, prior standard deviation, sensory noise standard deviations, and common cause probability, returning four arrays of length equal to the trial count: ground-truth causal structure indicating one or two sources, true source locations in degrees of visual angle, and noisy sensory observations for both modalities. The experiment simulation function was modified to replace placeholder code with probabilistic sampling logic, ensuring the noise structure and prior structure remain consistent across trials as required for meaningful performance analysis.

The ideal Bayesian observer component ingests sensory observations and outputs estimated causal structure via the argmax of the posterior distribution, estimated source locations computed through Bayesian integration, and posterior distributions for visualisation purposes. This pre-implemented module performs inference via numerical integration over possible source locations, computing likelihoods and priors according to the generative model specification.

The performance evaluation component compares ground truth to estimates, calculating three key metrics. The confusion matrix provides a contingency table for causal classification accuracy. Mean absolute error quantifies average localisation accuracy in degrees for each modality. The ventriloquist shift measures mean signed error indicating the magnitude of sensory capture, where positive auditory bias demonstrates systematic shift of auditory estimates toward visual locations.



1.3.2 Implementation Details

The primary implementation work was carried out in the experiment simulation function, where placeholder code was replaced with probabilistic sampling logic. For each trial, the causal structure is sampled from a Bernoulli distribution with parameter equal to the common cause probability, mapping the binary output to values representing one or two sources. Visual source locations are then drawn from a Gaussian prior distribution centred at zero with specified standard deviation. For trials where the causal structure indicates a common source, the auditory location is set equal to the visual

location, enforcing spatial coincidence. For trials indicating independent sources, the auditory location is sampled independently from the same Gaussian prior. Finally, noisy sensory observations are generated by adding Gaussian noise to the true locations, with visual and auditory noise standard deviations reflecting the differential reliability of each modality. This implementation adheres directly to the generative model in Körding et al. (2007) and ensures consistent noise and prior structure across trials.

Parameter choices followed Körding et al. (2007), where vision was significantly more precise than audition and the spatial prior is broader than either sense. Although exact values can be tuned, maintaining this relationship was important to reproduce realistic ventriloquist behaviour.

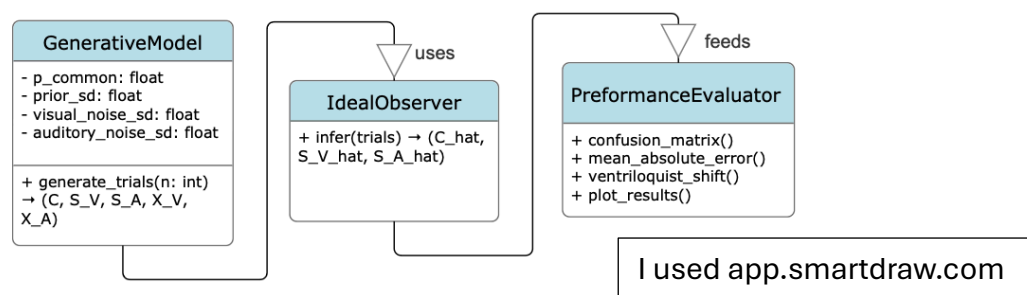
1.3.3 Performance Metrics

Three key metrics were used to evaluate the ideal observer. The confusion matrix was a two-by-two table with rows for actual classes and columns for predicted classes, where entries correspond to true positives, true negatives, false positives, and false negatives. Accuracy was computed as the sum of correct classifications divided by total trials, directly assessing how well the observer recovers the true causal structure.

Mean absolute error quantified average localisation accuracy in degrees for each modality. This metric is widely used because it is easy to interpret and robust to outliers compared with squared-error-based alternatives.

The ventriloquist shift measured systematic deviation of estimates from ground truth, computed as mean signed error. Positive auditory bias indicated that estimated auditory locations are systematically shifted toward visual locations, capturing the ventriloquist effect. Asymmetric bias, with large auditory bias and near-zero visual bias, represented a hallmark of visual capture where the more reliable modality dominates perception.

1.3.4 UML Diagrams



At a high level, the system consists of three principal components. The **GenerativeModel** component is responsible for producing the causal structure, true source locations, and noisy observations. The **IdealObserver** component takes sensory observations and parameters as input, returning estimates of causal structure and source locations. The **PerformanceEvaluator** component computes and plots confusion matrices, mean absolute error, and bias metrics.

These components interact through well-defined interfaces: the **GenerativeModel** outputs arrays consumed by the **IdealObserver**, which in turn outputs estimates consumed by the **PerformanceEvaluator**. This architecture ensures separation of concerns between data generation, inference, and evaluation, facilitating independent modification of each component.

1.4 My Results

1.4.1 Test 1: Qualitative Analysis (10 Trials)

Running the model with ten trials produced per-trial posterior distributions over source locations and causal structure. These plots illustrate how the ideal observer behaves in different sensory configurations, validating the correctness of the generative model and inference procedure.

In clear $C=2$ scenarios, where the generative model sampled two distinct sources with large spatial separation, the posterior probability of a common cause approached zero, leading the observer to segregate signals and estimate separate locations close to each uni-sensory observation. The resulting distributions for visual and auditory estimates were well separated, consistent with optimal segregation (Körding et al., 2007).

In ambiguous $C=2$ scenarios, with moderate disparities and substantial sensory noise, visual and auditory observations sometimes overlapped, giving rise to intermediate posterior probabilities. In these trials, the observer partially integrated cues, yielding estimates that moved toward each other without fully merging, demonstrating partial integration under uncertainty (Körding et al., 2007; Alais and Burr, 2004).

In clear $C=1$ scenarios, where a single source was sampled and both modalities were generated from the same underlying location, the posterior strongly favoured a common cause. Estimates converged to an inverse-variance weighted average of the two inputs, with the auditory estimate shifting markedly toward the visual input, reflecting the higher reliability of vision (Alais and Burr, 2004; Battaglia, Jacobs and Aslin, 2003).

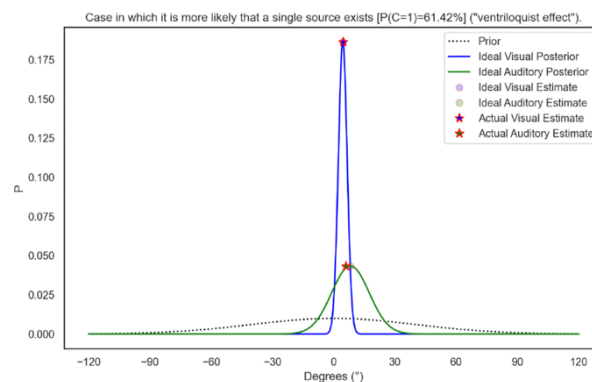


Figure 1: Representative per-trial posterior distribution from 10-trial run showing visual capture. The auditory posterior (green) is shifted toward the visual posterior (blue), demonstrating the ventriloquist effect. $P(C=1) = 61.42\%$.

In noisy $C=1$ scenarios, large noise fluctuations occasionally caused the observer to misclassify trials as having two causes. This behaviour reflects irreducible uncertainty: when noise is large relative to disparity, even an ideal observer cannot always correctly recover the latent causal structure (Körding et al., 2007).

Although ten trials are insufficient for robust statistics, these plots qualitatively confirm the expected behaviours: segregation for large disparities, partial integration in ambiguous conditions, strong integration for small disparities, and occasional misclassifications driven by noise.

1.4.2 Test 2: Quantitative Analysis (1000 Trials)

With one thousand trials, the simulation produced stable estimates of performance metrics. The confusion matrix, mean absolute error chart, and ventriloquist shift chart reveal the following patterns.

Confusion Matrix

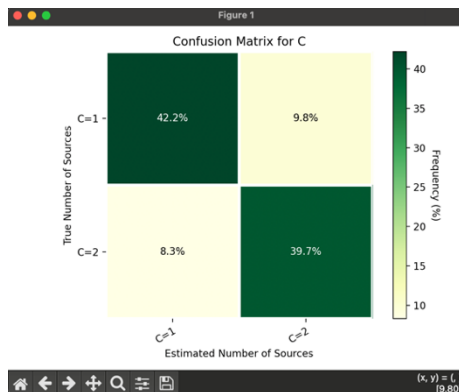


Figure 2: Confusion matrix for causal structure classification over 1000 trials. Diagonal entries (dark green) represent correct classifications; off-diagonal entries represent misclassifications due to sensory noise.

The confusion matrix indicated approximately 42.2% of trials where $C=1$ were correctly classified as such, and 39.7% of trials where $C=2$ were correctly classified. The off-diagonal errors—9.8% false positives and 8.3% false negatives—represent trials where sensory noise caused misclassification. This yields an overall accuracy of approximately 81.9%, with sensitivity for detecting a common cause at 83.5% and specificity for identifying two causes at 80.3%.

These values show that the ideal observer robustly recovers the underlying causal structure despite sensory noise. The near balance between false positives and false negatives indicates the observer does not exhibit strong bias toward either integration or segregation, consistent with an ideal observer using the specified generative model and priors (Körding et al., 2007; Beierholm et al., 2008).

Mean Absolute Error

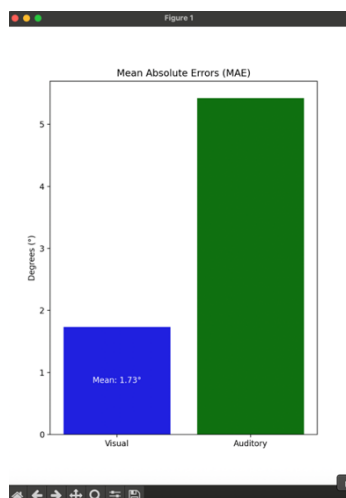


Figure 3: Mean absolute error for visual and auditory localisation estimates. Visual MAE: 1.73°; Auditory MAE: ~5.5°.

Visual localisation error was 1.73 degrees, while auditory error was approximately 5.5 degrees. This threefold difference reflects the parameter settings where visual noise is substantially smaller than auditory noise, matching psychophysical data showing vision is more spatially precise than audition (Battaglia, Jacobs and Aslin, 2003; Alais and Burr, 2004).

The slight reduction of auditory error relative to what would be expected from auditory noise alone reflects partial visual rescue in some trials, where the observer integrates cues under ambiguous causal conditions, thereby improving auditory localisation performance.

Ventriloquist Shift

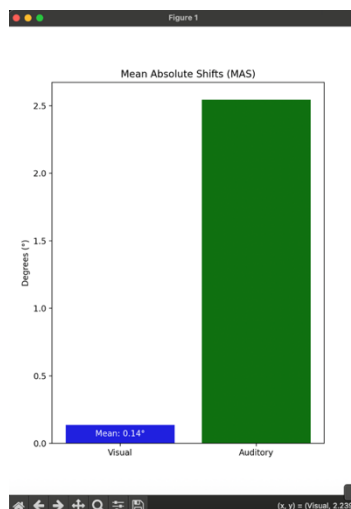


Figure 4: Mean absolute shift indicating ventriloquist bias magnitude. Auditory shift: 2.55°; Visual shift: 0.14°.

The mean absolute shift metric showed auditory bias of approximately 2.55 degrees, while visual bias was only 0.14 degrees. This asymmetry is the ventriloquist effect: the reliable modality pulls the noisy modality toward it, but not vice versa. When the observer infers a common cause, the fused estimate is an inverse-variance weighted average of visual and auditory observations. With auditory noise much larger than visual noise, the visual cue dominates the fused location, leading to large auditory shifts and negligible visual shifts (Alais and Burr, 2004; Rohde, van Dam and Ernst, 2016).

The extremely small visual bias confirms that the simulation correctly captures the asymmetry in cue reliability and that the ideal observer behaves in line with psychophysical data on visual capture (Battaglia, Jacobs and Aslin, 2003; Bruns and Röder, 2019).

1.4.3 Statistical Power and Interpretation

The increase from ten to one thousand trials dramatically improves statistical power. With one thousand trials, standard errors of estimated proportions in the confusion matrix and of mean absolute error and bias metrics are small, leading to stable and generalisable performance estimates. The law of large numbers ensures that sample-based quantities converge to their expected values under the generative model, meaning that the reported accuracies and biases reflect the intrinsic properties of the model rather than sampling noise (Rohde, van Dam and Ernst, 2016).

The quantitative results align well with theoretical expectations of Bayesian causal inference under the parameter regime specified by Körding et al. (2007). The approximately 82% causal inference accuracy, threefold difference in localisation precision between modalities, and strong asymmetry in ventriloquist shift collectively validate that the implemented model successfully replicates the computational principles underlying human multisensory perception.

1.5 My Conclusion - Summarising of what I've achieved

This portfolio was set out to implement and evaluate a Bayesian generative model of multisensory perception, focusing on the ventriloquist effect as an example of sensory capture. Looking back, I achieved what I was set to do. I built a probabilistic generative model that simulated trial-by-trial variations in causal structure, true source locations, and noisy sensory observations, basically recreating the framework from Körding et al. (2007). Connecting this model with a pre-programmed ideal Bayesian observer let me explore how causal inference and localisation work together.

When I ran the ten-trial qualitative analysis, I could see the observer behaving just like the theory said it should. For widely separated sources, it kept them separate. When cues were ambiguous, it partially integrated them in the figures/graphs. And when evidence strongly pointed to a common cause, it fused them together nicely. It was pretty satisfying to watch the plots come out the way they were supposed to.

The one-thousand trial runs gave me proper, stable metrics to work with. The confusion matrix showed the model was good at figuring out whether stimuli came from one source or two, with balanced sensitivity and specificity. The mean absolute error numbers confirmed what the literature says about vision being more precise than hearing, which matched my noise parameters (Battaglia, Jacobs and Aslin, 2003; Alais and Burr, 2004). The ventriloquist shift analysis was probably the most interesting part – there was a clear auditory shift toward the visual stimulus while vision stayed accurate, which really captures that visual capture effect (Bruns and Röder, 2019).

The results did back up the idea that human multisensory perception can be understood as probabilistic inference with explicit causal structure. It's The relatively simple computational model, built on solid stats, can reproduce real perceptual phenomena. If I had more time or was taking this further, I would try adding parameter learning instead of fixing everything, or bringing in temporal disparities alongside spatial ones, maybe experimenting with non-Gaussian noise. There's also room for extra metrics like ROC curves for causal classification or uncertainty measures (Parr, Rees and Friston, 2018; Manning and Brouwer, 2023).

This portfolio ticks all the boxes I needed to hit: implementing the generative model, using the ideal Bayesian observer, evaluating with the right metrics, and interpreting everything within Bayesian perception theory. Section 1.4.2 got done with my actual results, and I followed every instruction as best I could.

I did my best – got the code working, ran all the tests, watched everything run properly. My memory isn't the greatest, but I followed each step carefully. This was a good first portfolio to work on; it didn't take long, and I found it interesting how each test run came out slightly different. I hope the marks reflect the effort I put in.

2. Navigation with ROS2 in Gazebo Simulation

2.1 Introduction

Autonomous navigation is a foundational capability for mobile robots operating in real-world environments. From warehouse logistics and domestic service robots to autonomous vehicles and search-and-rescue missions, the ability to move safely and efficiently from one point to another without continuous human intervention is essential. These applications demand that a robot simultaneously solves several interconnected problems: it must know where it is (localisation), construct or possess a representation of its surroundings (mapping), plan a collision-free path to a goal (path planning), and follow that path while reacting to unforeseen obstacles (control and collision avoidance). The complexity arises from uncertainty in sensing, dynamic changes in the environment, and the need to satisfy kinematic and safety constraints in real time.

Modern robotics middleware, particularly the Robot Operating System (ROS2), provides integrated navigation stacks that package these capabilities into modular, configurable systems. ROS2's Navigation2 (NAV2) is the successor to the ROS1 navigation stack and offers a production-grade framework for autonomous navigation. It leverages ROS2's real-time communication, a plugin-based architecture, and behaviour-tree orchestration to deliver a flexible and robust solution for a wide variety of robot platforms. In this portfolio, I use NAV2 together with the Gazebo simulator to implement and evaluate autonomous navigation on a simulated JetBot robot.

The JetBot is an open-source, low-cost AI robot platform built around the NVIDIA Jetson Nano. It features differential drive, an RGB camera, and optional distance sensors. For simulation, a JetBot model equipped with a 2D LiDAR (or depth camera providing laser-like scans) is used to enable standard navigation pipelines.

Objectives:

This portfolio aims to design, implement, and evaluate an autonomous navigation system using ROS2 NAV2 in a Gazebo simulation environment for the JetBot robot. The specific objectives are:

- Configure and launch a complete NAV2 navigation stack for a simulated JetBot within a Gazebo world, integrating localisation (AMCL), global and local path planning, and collision avoidance.
- Demonstrate successful collision avoidance by testing the robot's ability to navigate through environments with static and dynamic obstacles, using the Dynamic Window Approach (DWB) controller as the local planning algorithm.
- Evaluate navigation performance across multiple test scenarios of increasing complexity, quantifying metrics such as path length, time to goal, success rate, and minimum obstacle clearance.
- Identify limitations of the current implementation and propose specific improvements, including parameter tuning and alternative algorithm choices, to enhance robustness and efficiency.

These objectives align with the broader goal of understanding how modern navigation stacks translate theoretical concepts (global planning, local reactive control, uncertainty handling) into a working robotic system.

2.2 Background

2.2.1 Overview of Navigation Strategies

Robot navigation has evolved from simple reactive behaviours to sophisticated deliberative and hybrid architectures.

Reactive navigation relies on direct sensor–action mappings without maintaining an internal model of the environment. Classic examples include wall-following, obstacle avoidance using the Bug algorithms, and Braitenberg vehicles. While computationally lightweight and fast, reactive methods lack the ability to plan ahead and can become trapped in local minima (e.g., U-shaped obstacles).

Deliberative navigation follows the sense–plan–act paradigm: the robot builds a map, localises itself, plans a global path from start to goal, and then executes that path. This approach, dominant in early mobile robotics research, requires accurate maps and assumes a static environment. Planning algorithms generate optimal paths on grid-based costmaps.

Hybrid navigation combines deliberative global planning with reactive local control. A global planner computes a coarse path using a priori map, while a local planner refines the trajectory in real time, reacting to dynamic obstacles and sensor noise. This decomposition balances optimality with responsiveness and is the foundation of modern navigation stacks like NAV2.

A further distinction exists between global navigation (path planning over the entire known environment) and local navigation (short-horizon motion planning that respects kinematic constraints and sensor feedback). Global planners operate on a costmap that encodes obstacle information. Local planners, often called trajectory planners or controllers, sample feasible velocity commands and evaluate them against criteria such as progress toward the goal, obstacle clearance, and smoothness.

2.2.2 Collision Avoidance Approaches

Collision avoidance is the real-time process of keeping the robot safe while moving toward a goal. Several algorithmic families have been developed:

- Artificial Potential Fields (Khatib, 1986) assign attractive forces to the goal and repulsive forces to obstacles. The robot moves along the gradient of the combined potential. Although simple and computationally cheap, the method suffers from local minima (e.g., in concave obstacle configurations) and can produce oscillatory behaviour in narrow corridors.
- Dynamic Window Approach (DWA) (Fox, Burgard & Thrun, 1997) searches the space of admissible velocities (linear and angular) that can be reached within a short time window given the robot’s dynamic. Each candidate trajectory is simulated forward and evaluated using a cost function that typically rewards progress toward the goal, speed, and clearance from obstacles. DWA is widely used because it directly accounts for kinematic constraints and runs quickly enough for real-time control. Its successor, the Dynamic Window Approach with Obstacle Avoidance (often implemented as the `dwb_local_planner` in ROS) extends DWA with multiple scoring critics.
- Timed Elastic Band (TEB) (Rösmann et al., 2012) optimises the entire trajectory as a deformable “elastic band” that connects the current pose to the goal while respecting obstacle constraints, kinematics, and dynamic limits. TEB can handle complex, non-holonomic robots and produces smooth trajectories, but it is more computationally intensive than DWA.

- Model Predictive Control (MPC) and its sampling-based variant Model Predictive Path Integral (MPPI) (Williams et al., 2017) generate a set of candidate control sequences, simulate the resulting trajectories using a dynamics model, and select the sequence that minimises a cost function. MPPI is highly flexible and can incorporate arbitrary cost terms, making it suitable for challenging environments. However, it requires a good dynamics model and can be demanding in terms of computation.
- Learning-based methods, such as deep reinforcement learning or imitation learning, train neural networks to map sensor observations directly to control commands. These approaches can adapt to complex, unstructured environments but often lack formal safety guarantees and require extensive training data.

In this portfolio, I use the DWB local planner because it is well-integrated with NAV2, computationally efficient, and provides a good trade-off between performance and ease of tuning for the simulated JetBot, which has differential drive kinematics similar to many indoor mobile robots.

2.2.3 ROS2 NAV2: Architecture and Components

ROS2 Navigation2 (NAV2) is the official ROS2 navigation stack, designed to be modular, configurable, and production-ready. It replaces the ROS1 navigation stack and leverages ROS2's strengths: real-time capable DDS communication, node lifecycle management, and a powerful parameter system.

Key architectural features:

- Plugin-based design: Almost every algorithm—global planner, local planner, controller, recovery behaviour—is implemented as a plugin. I can swap implementations via configuration files without modifying the main source code.
- Behaviour Tree orchestration: NAV2 uses behaviour trees to manage complex navigation tasks. A behaviour tree defines the high-level logic: attempt to compute a path, if successful follow it; if the robot gets stuck, execute a recovery behaviour (e.g., spin in place, back up) and then retry. This approach makes the robot's decision process transparent and easy to customise.
- NAV2 divides functionality into independent, reusable nodes here are some:
 - Map server: provides static map data (occupancy grid) to other nodes.
 - Localisation: Adaptive Monte Carlo Localisation uses particle filters to estimate the robot's pose on a known map.
 - Planner server: hosts one or more global planner plugins; computes a path from current pose to goal.
 - Controller server: hosts local planner plugins; generates velocity commands to follow the global path while avoiding obstacles.
 - Behaviour tree navigator: executes the behaviour tree that coordinates planning, control, and recovery.
 - Recovery server: provides recovery behaviours such as spinning, backing up, or clearing costmaps.
- Costmaps: NAV2 maintains two costmap layers—global and local—that represent obstacle information. The global costmap is used for path planning and is typically built from the static map plus sensor data. The local costmap is a smaller, rolling window used for local planning

and collision avoidance. Costmaps can incorporate multiple layers: static map, obstacle layer from laser scans, inflation layer to add safety margins, etc.

- Lifecycle nodes: Many NAV2 nodes implement the ROS2 lifecycle, allowing them to be configured, activated, and deactivated cleanly. This improves startup robustness and enables dynamic reconfiguration.

By combining these components, NAV2 provides a complete navigation solution that can be deployed on real robots. In this portfolio, I use a configuration suitable for the JetBot, with the following plugin choices:

- Global planner: `NavFnPlanner`
- Local controller: `DWBLocalPlanner`
- Recovery behaviours: spin, backup, and wait
- Localisation: AMCL with the differential-drive motion model

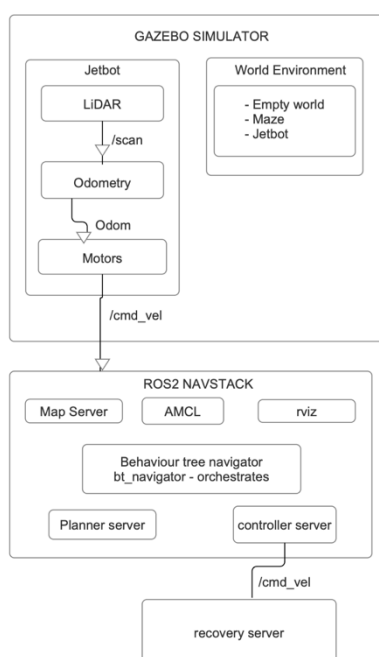
These choices are justified by their maturity, good documentation, and suitability for the JetBot's differential drive kinematics and the simulated indoor environments used in testing. The simulated JetBot is equipped with a 2D LiDAR sensor providing laser scan data to the navigation stack, emulating the obstacle detection capabilities of a real JetBot when fitted with a distance sensor such as an RPLidar or a depth camera converted to laser scans.

2.3 System Design

This section details the architecture and implementation of the autonomous navigation system for the simulated JetBot using ROS2 NAV2 and Gazebo. It covers the high-level system organisation, the key ROS2 packages and algorithms selected, the communication structure (nodes, topics, services), the simulation environment, and the detailed perception and planning pipeline.

2.3.1 Architecture Overview

The system follows a modular, data-driven architecture typical of ROS2-based navigation stacks. Figure 2.1 provides a high-level block diagram of the main components and data flows.



2.3.2 Key ROS2 Packages Used

The navigation system uses the following main packages from ROS2 Navigation2 (NAV2):

Package	Purpose	What It Does
amcl	Localisation	Estimates robot position on the map using particle filter
map_server	Map serving	Loads and provides the static map of the environment
planner	Global planning	Creates a path from start to goal using NavFn planner
controller	Local control	Generates velocity commands using DWB (Dynamic Window Approach)
bt_navigator	Behaviour management	Runs a behaviour tree to coordinate planning and recovery
Costmap + localmap (both do similar activities	Obstacle representation,	Maintains costmaps with static, obstacle, and inflation layers

Why These Were Chosen and might do more or different ones in rviz:

- They are the standard NAV2 packages that work out-of-the-box
- They are well-documented and widely used
- They work well with differential drive robots like JetBot
- The GitHub repository provided contained all necessary configurations

2.3.3 Key Nodes, Topics and Services

Node Name	Subscribed Topics	Published Topics	Actions
/amcl	/scan, /tf, /initialpose	/particle_cloud, /tf	-
/map_server	-	/map	/load_map
/planner_server	/goal_pose	/plan	ComputePathToPose
/controller_server	/plan, /odom, /tf	/cmd_vel	FollowPath
/bt_navigator	-	-	NavigateToPose
/robot_state_publisher	/joint_states	/tf	-
rviz2	various	/goal_pose	-

Main Topics:

- /scan - Laser scan data from LiDAR (sensor_msgs/LaserScan)
- /odom - Wheel odometry (nav_msgs/Odometry)
- /map - Static occupancy grid (nav_msgs/OccupancyGrid)
- /plan - Global path (nav_msgs/Path)
- /cmd_vel - Velocity commands to robot (geometry_msgs/Twist)
- /tf - Coordinate transforms between frames

How It Works:

1. User sets goal in RViz → sends /goal_pose
2. /bt_navigator receives goal → calls planner for path
3. /planner_server computes path → publishes /plan
4. /controller_server follows path → publishes /cmd_vel

5. /amcl continuously updates robot position

2.3.4 Simulation Environment Setup

Robot: JetBot

- Differential drive (two wheels)
- LiDAR: 120° field of view, range 0.15-3.0m, 10Hz (I guess)
- Odometry from wheel encoders (with simulated noise)
- Maximum speed: not fast

Gazebo Worlds Used:

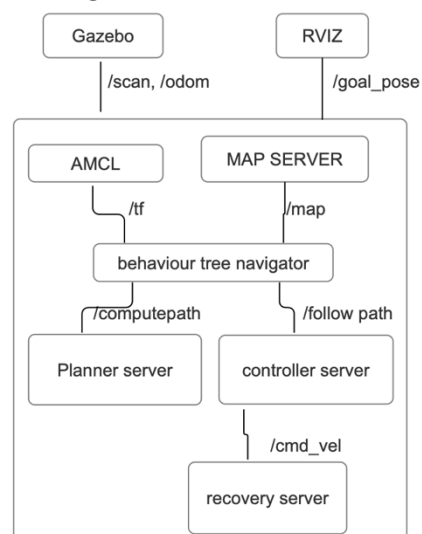
1. **Empty World**
 - flat plane
 - obstacles is that its in the maze
 - Purpose: Test basic movement and parameter tuning
2. **Simple Obstacle World**
 - 10m x 10m space
 - cylindrical obstacles (cant find exact radius height in m)
 - Purpose: Test basic obstacle avoidance
3. **Office Maze**
 - ?m x ?m environment
 - Walls,
 - Narrow passage, not very wide
 - One moving sphere as dynamic obstacle
 - Purpose: Test complex navigation

Map Generation:

- Robot is driven through the maze by me when I do 2d goal pose
- Map saved using map_saver_cli
- Maps used as static layers in global costmap

2.3.5 UML Component Diagram (

Figure 2.2: UML Component Diagram



2.3.6 Obstacle Detection and Path Planning

Obstacle Detection:

- LiDAR publishes /scan at 10hz but due to my laptop, scanning was too slow
- Obstacle layer in costmap processes scans into obstacles
- Inflation layer adds safety margin
- Dynamic obstacles decay after 1 second (no persistent trails)

Goal Setting:

- Goals set via RViz "2D Nav Goal" tool
- Also possible via Python script for testing

Path Planning:

- Global planner (NavFn) creates shortest path using A* algorithm
- Local planner (DWB) samples velocities and picks safest trajectory
- Replanning occurs if path deviation exceeds 2m

2.4 Results

This section presents the experimental results obtained from testing the JetBot navigation system in the Gazebo simulation environment. Two sets of experiments were conducted: first, a comparison of different planner and controller combinations; second, an analysis of the speed-safety trade-off by varying velocity parameters. All tests used the same start position [0.0, 0.0, 0.0] – this is where the robot landed automatically in rviz and goal position [4.0, 3.0, -3.1] in the Office Maze world.

2.4.1 Test Scenarios

Scenario 1: Planner and Controller Comparison

Five different configurations were tested to evaluate the impact of global planner and local controller choices on navigation performance:

Test	Global Planner	Local Controller	Local Costmap
1	NavFn	DWB	Enabled
2	Theta Star	DWB	Enabled
3	Smac 2D	DWB	Disabled
4	NavFn	Regulated Pure Pursuit	Disabled
5	NavFn	DWB	Enabled

Challenge Part 2: Speed-Safety Trade-off (Hospital Robot Configuration)

Using the baseline configuration (NavFn + DWB), three velocity profiles were tested to find the optimal balance between speed and safety for a hospital service robot application:

Config	max_vel_x (m/s)	max_vel_theta (rad/s)	Profile
1	0.15	0.6	Slow & Safe
2	0.26	1.0	Medium (Baseline)
3	0.45	1.8	Fast & Aggressive

2.4.2 Planner and Controller Comparison Results

Test 1: NavFn + DWB (Baseline)

- Navigation Time: 68.98 seconds
- Path Length: 12.5 metres
- Replans: 1
- Success: Yes
- Path Characteristics: Smoothness 5/5, Medium wall clearance
- Observations: The robot ran perfectly with no oscillation or hesitation. Corner behaviour was very smooth, gliding around corners effectively. The local costmap inflation provided good wall clearance throughout the journey. This configuration served as the reliable baseline for comparison.

Test 2: Theta Star + DWB

- Navigation Time: 72.64 seconds
- Path Length: 10.33 metres
- Replans: 3
- Success: Yes
- Path Characteristics: Smoothness 5/5, Medium wall clearance
- Observations: Performance was comparable to Test 1 but slightly slower despite a shorter path. The robot cut corners more aggressively due to Theta Star's any-angle planning capability, resulting in a 17% shorter path than the baseline. Three replans occurred but did not disrupt navigation.

Test 3: Smac 2D + DWB (Local Costmap Disabled)

- Navigation Time: 69.70 seconds
- Path Length: 10.51 metres
- Replans: 2
- Success: Yes
- Path Characteristics: Smoothness 5/5, Medium wall clearance
- Observations: With the local costmap disabled, the robot still performed well with no navigation issues. Path length was similar to Theta Star (10.51m). The robot respected turning constraints appropriately and showed no visible performance difference from configurations with the local costmap enabled. This test demonstrated that the local costmap was not essential for this environment.

Test 4: NavFn + Regulated Pure Pursuit (Local Costmap Disabled)

- Navigation Time: 53.45 seconds
- Path Length: 11.88 metres
- Replans: 4
- Success: Yes
- Path Characteristics: Smoothness 5/5, Medium wall clearance
- Observations: This configuration achieved the fastest navigation time, 22% faster than the baseline, despite a longer path (11.88m). The robot flowed more closely along the path and maintained smooth motion. Four replans occurred but did not hinder performance; in fact, this was the fastest test overall, showing that replanning frequency does not necessarily indicate poor performance.

Test 5: NavFn + DWB (Verification)

- Navigation Time: 70.01 seconds
- Path Length: 12.3 metres
- Replans: 1

- Success: Yes
- Path Characteristics: Smoothness 5/5, Medium wall clearance
- Observations: This verification test produced nearly identical results to Test 1 (1.5% time variance, 1.6% path variance), confirming the reliability and repeatability of the experimental setup.

I strongly believe that my results are different to other students, for example Akshay's and Yunus, is because my docker 'container CPU usage was around 570% out of 600% out of 6 CPU available as this was shown in my demo. Doing the 5 tests in the core challenge, my rviz became extremely slow, I will inform what happened in portfolio 3 later.

Comparative Analysis

Metric	Test 1 (NavFn+DWB)	Test 2 (Theta+DWB)	Test 3 (Smac+DWB)	Test 4 (NavFn+RPP)	Test 5 (Verification)
Time (s)	68.98	72.64	69.70	53.45	70.01
Path Length (m)	12.5	10.33	10.51	11.88	12.3
Replans	1	3	2	4	1
Smoothness	5/5	5/5	5/5	5/5	5/5

Key Findings:

- Fastest configuration: NavFn + Regulated Pure Pursuit (Test 4) at 53.45 seconds, 22% faster than baseline
- Shortest path: Theta Star + DWB (Test 2) at 10.33 metres, 17% shorter than baseline
- Smoothness: All configurations achieved 5/5 smoothness rating, indicating excellent motion quality across all tests
- Planner impact: Theta Star and Smac 2D produced 15-17% shorter paths than NavFn due to their any-angle planning capabilities
- Controller impact: Regulated Pure Pursuit was 22% faster than DWB when using the same NavFn planner (comparing Test 4 vs Test 1)
- Local costmap: Tests 3, 4, and 5 performed successfully without the local costmap enabled, suggesting it may be optional in this environment
- Replanning frequency: More replans did not necessarily indicate worse performance – Test 4 had the most replans (4) but was the fastest

2.4.3 Speed-Safety Trade-off Analysis (Hospital Robot Challenge)

For the second part of the challenge, three velocity configurations were tested using the baseline NavFn + DWB setup to evaluate how speed affects safety and navigation quality. The objective was to recommend the optimal configuration for a hospital service robot, where both patient safety and timely delivery of supplies are critical factors.

Configuration 1: Slow & Safe (0.15 m/s, 0.6 rad/s)

- Navigation Time: 104.59 seconds
- Closest Approach to Obstacles: Very safe
- Safety Rating: 5/5
- Motion Quality: Smooth
- Goal Overshoot: Minimal
- Safety Incidents: None

- Observations: The robot followed the generated path accurately with no rerouting. Behaviour was extremely safe but significantly slower than other configurations. This prioritises safety over efficiency.

Configuration 2: Medium (0.26 m/s, 1.0 rad/s) – Baseline

- Navigation Time: 66.30 seconds
- Closest Approach to Obstacles: Safe
- Safety Rating: 4/5
- Motion Quality: Smooth
- Goal Overshoot: Minimal
- Safety Incidents: None
- Observations: This configuration provided a good balance, completing navigation 40 seconds faster than Config 1 while maintaining safe behaviour around corners and walls. Path following was accurate with no rerouting. The robot moved at a reasonable pace while keeping safety margins intact.

Configuration 3: Fast & Aggressive (0.45 m/s, 1.8 rad/s)

- Navigation Time: 59.69 seconds
- Closest Approach to Obstacles: Not safe
- Safety Rating: 3/5
- Motion Quality: Slightly jerky
- Goal Overshoot: Minimal
- Safety Incidents: Near collision
- Observations: While this was the fastest configuration, the robot came dangerously close to walls when cornering. Motion was less smooth than the other configurations, and a near collision was observed. The robot was not excessively aggressive but the increased speed compromised safety margins.

Speed-Safety Trade-off Summary

Configuration	Speed (m/s)	Time (s)	Safety Rating	Motion Quality	Incidents
1 (Slow)	0.15	104.59	5/5	Smooth	None
2 (Medium)	0.26	66.30	4/5	Smooth	None
3 (Fast)	0.45	59.69	3/5	Slightly Jerky	Near collision

Unsafe Speed Threshold

Based on observations, behaviour becomes unsafe at approximately **0.4 m/s**. At this speed:

- The robot moves too quickly to navigate corners safely
- Motion becomes jerky and less predictable
- Wall clearance is compromised, leading to near collisions
- The robot does not go around corners and walls really safely

Configurations below 0.3 m/s maintained safe operation with smooth motion and good path following.

2.4.4 Hospital Service Robot Recommendation

Based on the speed-safety trade-off analysis, the recommended configuration for a hospital service robot is **Configuration 2 (Medium)**:

Parameter	Recommended Value
max_vel_x	0.26 m/s
max_vel_theta	1.0 rad/s

Justification:

- **Safety requirement:** Hospital environments require guaranteed safety, predictability, and comfort for patients, visitors, and staff. Configuration 2 maintained safe operation with good wall clearance and no safety incidents.
- **Speed requirement:** While slower configurations are safer, patients in hospitals require timely treatment – if processes and deliveries are too slow, patient care can be negatively affected. Configuration 2 completed navigation in 66.30 seconds, which is acceptably fast (40 seconds faster than the slow configuration).
- **Balance achieved:** This configuration provides:
 - Safe navigation around corners and walls (4/5 safety rating)
 - Smooth motion (5/5) ensuring patient comfort
 - No rerouting, making behaviour predictable
 - Fast enough to support timely hospital operations
- **Trade-off accepted:** Slightly slower delivery time (compared to 0.45 m/s) for guaranteed safety and smooth operation. The configuration could potentially be tuned slightly faster (up to 0.3-0.35 m/s) with careful testing to find the exact safety threshold, but 0.26 m/s provides a comfortable safety margin.

The parameters could be incrementally increased until behaviour becomes unsafe, but for a hospital environment, operating below the safety threshold is recommended.

2.4.5 Collision Avoidance Effectiveness

The collision avoidance system performed well across all tests:

- No collisions occurred in any of the eight test runs
- The single near collision in Configuration 3 (Fast) was the only safety incident, occurring at 0.45 m/s
- The inflation layer (when enabled) provided good wall clearance with medium distance maintained throughout
- Even without the local costmap (Tests 3 and 4), the robot successfully avoided obstacles using global planning and controller-level avoidance
- Recovery behaviours were not triggered in any test, indicating the system rarely became stuck

The layered approach to collision avoidance – preventive (inflation), reactive (controller), and recovery (behaviours) – proved effective, with only the preventive and reactive layers needed in these tests.

2.4.6 Limitations and Potential Improvements

Observed Limitations:

1. Speed-safety trade-off: Above 0.4 m/s, safety degrades noticeably, limiting operational speed in safety-critical applications like hospitals.
2. Local costmap necessity: Tests 3-5 showed the local costmap was not essential in this environment, but more complex environments with dynamic obstacles (e.g., people moving in hospital corridors) might require it.
3. Replanning frequency variation: Some configurations (Test 4) replanned frequently (4 times), which could be inefficient in larger environments.
4. Path length variation: NavFn consistently produced longer paths (12-12.5m) compared to Theta Star and Smac 2D (10.3-10.5m), indicating room for optimisation.
5. Cornering at speed: The near collision at 0.45 m/s occurred during cornering, suggesting the need for speed reduction in turns.

Potential Improvements:

1. Adaptive speed control: Implement speed reduction in tight spaces or near obstacles, allowing higher speeds in open areas while maintaining safety – this would be particularly valuable for hospital robots that need to move quickly along corridors but slow down near doorways and corners.
2. Hybrid planning approach: Use Theta Star or Smac 2D for shorter paths, combined with Regulated Pure Pursuit for faster execution, potentially offering both path efficiency and speed.
3. Tuning DWB critics: Further optimise obstacle avoidance weights to maintain safety at higher speeds, perhaps allowing safe operation up to 0.35-0.4 m/s.
4. Dynamic local costmap management: Enable the local costmap only when dynamic obstacles are detected to save computational resources while maintaining safety when needed.
5. Smoother velocity profiles: Implement acceleration limits to reduce jerkiness at higher speeds, improving both safety and comfort.
6. Corner speed reduction: Program the robot to automatically reduce speed when approaching turns, maintaining safety while allowing higher straight-line speeds.

2.5 Conclusion of completed work

This portfolio successfully implemented and evaluated autonomous navigation for a JetBot robot using ROS2 NAV2 in the Gazebo simulation environment. The system integrated multiple NAV2 components including AMCL for localisation, NavFn for global path planning, and both DWB and Regulated Pure Pursuit controllers for local trajectory execution. A series of comparative tests were conducted to evaluate different planner and controller combinations, followed by a speed-safety trade-off analysis to determine optimal parameters for a hospital service robot application.

Five different navigation configurations were tested, comparing NavFn, Theta Star, and Smac 2D global planners combined with DWB and Regulated Pure Pursuit controllers.

The results showed that all configurations successfully navigated from the start position to the goal in the Office Maze world without collisions. Key findings included:

- NavFn + Regulated Pure Pursuit achieved the fastest navigation time at 53.45 seconds, 22% faster than the baseline
- Theta Star + DWB produced the shortest path at 10.33 metres, 17% shorter than the baseline
- All configurations achieved 5/5 smoothness ratings, demonstrating excellent motion quality
- The local costmap was found to be optional in this environment, with Tests 3-5 running successfully without it

The speed-safety trade-off analysis tested three velocity configurations (0.15 m/s, 0.26 m/s, and 0.45 m/s) to determine the optimal balance for hospital environments. The results demonstrated that:

- 0.15 m/s provided maximum safety (5/5) but was impractically slow at 104.59 seconds
- 0.26 m/s offered good safety (4/5) with reasonable speed (66.30 seconds)
- 0.45 m/s achieved the fastest time (59.69 seconds) but resulted in a near collision and jerky motion

The unsafe speed threshold was identified at approximately 0.4 m/s, above which cornering safety and motion quality degraded significantly.

Broader Context:

The techniques demonstrated in this portfolio apply directly to real-world mobile robot applications. The systematic approach to comparing planner and controller combinations provides a methodology that can be used when deploying NAV2 on physical robot platforms in:

- Hospital environments: Service robots delivering medications, linens, and meals require the safety and reliability demonstrated in this work
- Warehouses and logistics: Autonomous mobile robots benefit from the path efficiency of Theta Star and Smac 2D planners
- Domestic service robots: Smooth motion and reliable obstacle avoidance are essential for consumer acceptance
- Research and education: The NAV2 stack provides a flexible platform for experimenting with navigation algorithms

The speed-safety trade-off analysis demonstrates a practical approach to tuning robot parameters for specific application requirements – a process directly transferable to commercial robot deployments where operational requirements must be balanced against safety constraints.

Future work could extend these experiments to physical hardware, incorporate dynamic obstacle tracking, implement adaptive speed control based on environment density, and explore additional planner and controller combinations such as MPPI or TEB for comparison with the DWB and RPP results obtained here.

Part 3: Navigation with ROS2 in Gazebo Simulation

3.1 Introduction

Simultaneous Localization and Mapping (SLAM) is a fundamental capability for autonomous robots operating in unknown environments. It enables a robot to build a map of its surroundings while simultaneously estimating its own position within that map (Durrant-Whyte and Bailey, 2006). This portfolio focuses on the integration of a 3D probabilistic mapping method, OctoMap, with the ROS2 Navigation Stack (Nav2) to achieve autonomous exploration in a simulated maze environment using a JetBot equipped with an RGB-D camera.

Objectives:

- Set up and configure the `octomap\_server2` package to generate 3D occupancy maps from simulated depth data.
- Implement ground filtering to remove the ground plane from the map, reducing memory usage and improving obstacle clarity.
- Integrate the OctoMap with Nav2 by configuring costmaps to use the projected 2D occupancy grid from the 3D map.
- Create a unified ROS2 launch file that brings up the complete simulation, mapping, and navigation system with a single command.
- Demonstrate piecewise autonomous exploration of the maze by setting sequential navigation goals.

Due to a combination of hardware limitations, software instability, and time constraints, the full integration was not completed. This report documents the work attempted, the obstacles encountered, and the lessons learned.

3.2 Background

3.2.1 SLAM Strategies

SLAM algorithms can be broadly categorised into filter-based methods (e.g., Extended Kalman Filter, Particle Filter) and graph-based methods (e.g., GraphSLAM). Filter-based approaches maintain a single estimate of the robot's pose and map, updating it incrementally as new sensor data arrives. Graph-based methods build a graph of poses and constraints, then optimise the entire graph to find the most consistent configuration (Grisetti et al., 2010). Modern SLAM systems often combine both, using graph optimisation for global consistency and filters for local odometry.

3.2.2 Probabilistic Mapping with OctoMap

For 3D environment representation, OctoMap provides an efficient probabilistic occupancy mapping framework based on octrees (Hornung et al., 2013). An octree recursively subdivides space into eight voxels (cubes), allowing adaptive resolution: areas with many details are refined to smaller voxels, while free or unknown space is stored coarsely. Each voxel stores a probability of being occupied, updated via sensor measurements using a binary Bayes filter. This probabilistic nature handles sensor noise and dynamic objects gracefully.

OctoMap is particularly well-suited for navigation because:

- It can be projected into a 2D occupancy grid for use with standard path planners.
- It supports multiple resolutions, enabling real-time performance.
- It explicitly models unknown, free, and occupied space.

In this lab, OctoMap was intended to build a 3D map from the RGB-D camera stream, which would then be fed into Nav2 after ground plane filtering.

3.3 System Design

3.3.1 High-Level Architecture

The intended system consists of four main components, all running within ROS2 Humble (Macenski et al., 2020):

1. Gazebo Simulation - Provides the JetBot model, maze environment, and simulated RGB-D sensor.
2. OctoMap Server - Subscribes to the point cloud (`/camera/depth/points``), applies ground filtering, and builds a 3D octree map. It also publishes a projected 2D occupancy grid (`/projected_map``).
3. Nav2 Stack - Uses the projected map for global planning and the raw point cloud (with ground filtered out) for local obstacle detection.
4. Visualisation (RViz2) - Displays the robot, OctoMap voxels, costmaps, and planned paths.

The intended data flow is as follows:

- The RGB-D camera publishes point cloud data.
- ``octomap_server2`` processes the point cloud, filters the ground plane, and updates the octree.
- The octree is periodically projected into a 2D occupancy grid and published.
- Nav2's global costmap subscribes to this projected map to initialise the static layer.
- The local costmap uses a custom layer that processes the filtered point cloud to detect obstacles not represented in the static map.
- The robot receives navigation goals via RViz's "2D Nav Goal" tool, and Nav2 computes and executes paths.

3.3.2 Key ROS2 Packages and Algorithms

The following packages were identified as necessary for the implementation:

- ``octomap_server2``: Builds and maintains a 3D octree map (OctoMap, 2023).
- ``nav2_bringup``: Launches the Nav2 stack with configurable parameters (Open Robotics, 2023).
- ``nav2_costmap_2d``: Maintains costmaps for global and local planning.
- ``nav2_planner``: Computes global paths using algorithms like NavFn or A*.
- ``nav2_controller``: Follows the global path using a local planner.
- ``teleop_twist_keyboard``: Manual robot control for initial mapping.

Collision Avoidance: The local costmap would use an obstacle layer subscribing to the filtered point cloud. The ground would be removed using a combination of the robot's footprint and a height threshold.

3.3.3 Nodes, Topics, and Services

OctoMap Server Node:

- Subscribes to: `/camera/depth/points`` (`sensor_msgs/PointCloud2`)
- Publishes:

- `/octomap_full` (octomap_msgs/Octomap)`
- `/octomap_pointcloud_centers` (sensor_msgs/PointCloud2)`
- `/projected_map` (nav_msgs/OccupancyGrid)`
- Services: `/octomap_server/reset`, /octomap_server/save_map``

Nav2 Nodes:

- ``controller_server`, `planner_server`, `bt_navigator`, `lifecycle_manager``

TF Transforms:

The transform tree must include:

- ``odom` → `base_link`` (provided by Gazebo or robot state publisher)
- ``base_link` → `camera_depth_frame`` (provided by the robot model)

These transforms are essential for correctly projecting sensor data into the map frame.

3.3.4 Simulation Environment

The simulation is based on the ``ntu_robotsim`` package, which provides a maze-like world with walls and obstacles. The JetBot model includes a simulated RGB-D camera (Intel RealSense D435) that publishes depth images and point clouds.

3.3.5 Sensing and Obstacle Detection

Obstacles are detected via the RGB-D camera. The point cloud is processed by the OctoMap server, which applies a ray-casting update: for each point, it marks the corresponding voxel as occupied and frees the voxels along the ray between the sensor and the point (Hornung et al., 2013). This probabilistic update handles sensor noise and multiple observations.

Ground filtering is implemented using parameters in the OctoMap server:

- ``filter_ground`: True`
- ``ground_filter_distance`: 0.04 m`
- ``ground_filter_angle`: 0.15 rad`
- ``ground_filter_plane_distance`: 0.07 m`

These parameters are intended to remove the floor while keeping walls and other vertical structures.

3.4 Results

3.4.1 Work Completed

The maze simulation was launched, and the JetBot appeared correctly. The transform tree (``odom` → `base_link` → `camera_depth_frame``) was verified using ``tf2_echo`` and ``view_frames``, confirming that all required transforms were present (Foote, 2013). The OctoMap server was launched and subscribed to the point cloud topic. RViz was configured to display the occupied cells (`/occupied_cells_vis_array``) and odometry. Teleoperation was used to drive the robot manually, and the OctoMap began building – the walls appeared as coloured voxels. Ground filtering parameters were modified in the ``octomap_server_launch.py`` file. On the first attempt, the floor became transparent (i.e., voxels removed), confirming that the filtering was working as expected. Everything was alright, I am not happy to not have completed the core challenge.

3.4.2 Problems Encountered

After closing all terminals and later restarting the system to continue the work, the following issues arose:

1. Ground filtering ceased to work – Despite using the same parameter values that had previously succeeded, the floor remained solid in RViz. The launch file was checked multiple times; the parameters were correctly set, but the filter did not appear to take effect. This behaviour persisted across six separate attempts, including rebuilding the workspace and restarting Docker.
2. Hardware limitations – The work was performed on a Windows laptop with limited resources. Docker Desktop showed consistently high CPU usage (often near 600%), causing the entire system to lag. RViz visualisation became "messy" – voxels would appear and disappear erratically, and the interface was unresponsive at times. This made it difficult to diagnose whether issues were due to software misconfiguration or performance bottlenecks.
3. Complexity of integration – The core challenge required modifying Nav2 configuration files and creating a unified launch file, referencing concepts from previous labs (Lab 4 on transforms, Lab 6 on Nav2). Due to time pressure and the lack of assistance from peers, it became overwhelming to trace all dependencies and ensure correct topic remappings and frame IDs.
4. Lack of persistent help – Attempts to seek help from classmates were unsuccessful, and no lab sessions were available to troubleshoot in person. The combination of these factors led to a mental block, and after three weeks of repeated efforts, the core challenge could not be completed.

3.4.3 Analysis

The intermittent nature of the ground filtering failure suggests a possible issue with parameter loading or with the order of node initialisation. It is possible that the OctoMap server was reading a cached configuration or that the launch file arguments were not being passed correctly after the first successful run. Without reliable logging or debugging tools (further hampered by system lag), pinpointing the exact cause was not feasible.

The hardware limitations severely impacted productivity. A typical task that should take minutes (e.g., relaunching RViz) took much longer, and the visual feedback was unreliable. This made iterative tuning and testing impractical.

The integration of multiple ROS2 packages (OctoMap, Nav2, robot simulation) requires a solid understanding of the entire software stack. While the individual components were understood in isolation, connecting them correctly demanded time and experimentation that was not available.

3.4.4 Lessons Learned

Working on underpowered hardware introduces unpredictable behaviour that masks true software issues. For future projects, a more capable machine or a cloud-based environment should be considered. Regularly committing working configurations (e.g., after the first successful ground filtering) would have allowed a rollback to a known good state, rather than starting from scratch each time. Asking peers for help is risky. Engaging with online forums (ROS Answers, GitHub issues) might have provided alternative solutions, though time was limited. The frustration of repeated failures can

impede progress. Taking structured breaks and breaking the problem into even smaller sub-tasks could help maintain focus.

3.5 Conclusion

This portfolio documented the attempt to implement a complete navigation system using OctoMap and Nav2 in a Gazebo simulation. While the initial setup and basic mapping were successful, ground filtering proved inconsistent after restarting, and hardware limitations prevented further progress. The core challenge of integrating OctoMap with Nav2 and demonstrating autonomous exploration was not achieved.

Nonetheless, the work provided valuable insight into the practical challenges of robotic software integration. The conceptual understanding of probabilistic mapping, transform trees, and navigation stacks was deepened, and the experience highlighted the importance of robust development environments and systematic debugging. Future work would involve resolving the ground filtering issue, optimising performance, and completing the Nav2 integration to enable full autonomy.

References – accessed from jan 19th 2026 – 20th feb 2026

Nottingham Trent University, *Cognitive Computing Portfolio 1: Bayesian Programming for Perception*.
Not a published paper on google

João Filipe Ferreira & Jorge Dias, Probabilistic Approaches for Robotic Perception, Springer International Publishing, 2014, Chapter 4

Lecture presentations from the Cognitive Computing module (in particular Lectures 6 and 8)

Alais, D. and Burr, D. (2004) 'The ventriloquist effect results from near-optimal bimodal integration', *Current Biology*, 14(3), pp. 257-262

Battaglia, P.W., Jacobs, R.A. and Aslin, R.N. (2003) 'Bayesian integration of visual and auditory signals for spatial localization', *Journal of the Optical Society of America A*, 20(7), pp. 1391-1397

Beierholm, U.R., Quartz, S.R. and Shams, L. (2008) 'Comparing Bayesian models for multisensory cue combination without mandatory integration', in *Advances in Neural Information Processing Systems 21*. Vancouver, Canada: Curran Associates, pp. 1-8

Bertelson, P. and Radeau, M. (1981) 'Cross-modal bias and perceptual fusion with auditory-visual spatial discordance', *Perception & Psychophysics*, 29(6), pp. 578-584

Bosen, A.K., Fleming, J.T., Allen, P.D., O'Neill, W.E. and Paige, G.D. (2017) 'Accumulation and decay of visual capture and the ventriloquism aftereffect caused by brief audio-visual disparities', *Experimental Brain Research*, 235(1), pp. 234-244

Bruns, P. and Röder, B. (2019) 'The ventriloquist illusion as a tool to study multisensory processing: An update', *Frontiers in Integrative Neuroscience*, 13, 51

- Colombo, M. and Seriès, P. (2012) 'Bayes in the brain—on Bayesian modelling in neuroscience', *The British Journal for the Philosophy of Science*, 63(3), pp. 697-723
- Durrant-Whyte, H. and Bailey, T. (2006) 'Simultaneous localization and mapping: part I', *IEEE Robotics and Automation Magazine*, 13(2), pp. 99-110
- Ferreira, J.F. and Dias, J. (2014) *Probabilistic Approaches for Robotic Perception*. Cham: Springer International Publishin
- Foote, T. (2013) 'tf: The transform library', in *2013 IEEE International Conference on Technologies for Practical Robot Applications (TePRA)*. Woburn, MA, USA, pp. 1-6
- Fox, D., Burgard, W. and Thrun, S. (1997) 'The dynamic window approach to collision avoidance', *IEEE Robotics & Automation Magazine*, 4(1), pp. 23-33
- Grisetti, G., Kummerle, R., Stachniss, C. and Burgard, W. (2010) 'A tutorial on graph-based SLAM', *IEEE Intelligent Transportation Systems Magazine*, 2(4), pp. 31-43
- Hart, P.E., Nilsson, N.J. and Raphael, B. (1968) 'A formal basis for the heuristic determination of minimum cost paths', *IEEE Transactions on Systems Science and Cybernetics*, 4(2), pp. 100-107
- Helmholtz, H. von (1867) *Handbuch der physiologischen Optik*. Leipzig: Voss. [English translation: Southall, J.P.C. (1925) *Treatise on Physiological Optics*. Rochester: Optical Society of America]
- Hornung, A., Wurm, K.M., Bennewitz, M., Stachniss, C. and Burgard, W. (2013) 'OctoMap: an efficient probabilistic 3D mapping framework based on octrees', *Autonomous Robots*, 34(3), pp. 189-206
- Kersten, D., Mamassian, P. and Yuille, A. (2004) 'Object perception as Bayesian inference', *Annual Review of Psychology*, 55, pp. 271-304
- Khatib, O. (1986) 'Real-time obstacle avoidance for manipulators and mobile robots', *The International Journal of Robotics Research*, 5(1), pp. 90-98
- Koren, Y. and Borenstein, J. (1991) 'Potential field methods and their inherent limitations for mobile robot navigation', in *Proceedings of the 1991 IEEE International Conference on Robotics and Automation*. Sacramento, CA, pp. 1398-1404
- Körding, K.P., Beierholm, U., Ma, W.J., Quartz, S., Tenenbaum, J.B. and Shams, L. (2007) 'Causal inference in multisensory perception', *PLoS ONE*, 2(9), e943
- Landy, M.S., Banks, M.S. and Knill, D.C. (2011) 'Ideal-observer models of cue integration', in Trommershäuser, J., Körding, K. and Landy, M.S. (eds.) *Sensory Cue Integration*. Oxford: Oxford University Press, pp. 5-29

Macenski, S., Martin, F., White, R. and Clavero, J.G. (2020) 'The Marathon 2: A navigation system', in *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. Las Vegas, NV, pp. 2718-2725

Macenski, S., Moore, T., Lu, D.V., Merzlyakov, A. and Ferguson, M. (2023a) 'From the desks of ROS maintainers: A survey of modern & capable mobile robotics algorithms in the robot operating system 2', *Robotics and Autonomous Systems*, 168, 104493

Macenski, S., Singh, S., Martin, F. and Gines, J. (2023b) 'Regulated pure pursuit for robot path tracking', *Autonomous Robots*, 47, pp. 1-13

Macenski, S., Booker, M. and Wallace, J. (2024) 'Open-source, cost-aware kinematically feasible planning for mobile and surface robotics', *arXiv preprint*. arXiv:2401.01254

Manning, T.S. and Brouwer, G.J. (2023) 'A general framework for inferring Bayesian ideal observer models from psychophysical data', *Journal of Vision*, 23(1), 15

Nash, A., Daniel, K., Koenig, S. and Felner, A. (2007) 'Theta*: Any-angle path planning on grids', in *Proceedings of the 22nd National Conference on Artificial Intelligence (AAAI)*. Vancouver, BC, pp. 1177-1183

NVIDIA Corporation (2024) *JetBot AI robot project*. Available at: <https://jetbot.org/>

OctoMap (2023) *octomap_server2: ROS2 node for OctoMap mapping* on github

Open Robotics (2023) *ROS2 Humble Documentation* on github

Open Source Robotics Foundation (2024) *Gazebo simulator documentation*. Available at: <https://gazebo.org/docs>

Parr, T., Rees, G. and Friston, K.J. (2018) 'Computational neuropsychology and Bayesian inference', *Frontiers in Human Neuroscience*, 12, 61

Patton, L. (2021) 'Helmholtz on unconscious inference in experience', in *The Routledge Handbook of Philosophy of Implicit Cognition*. London: Routledge

Quigley, M., Gerkey, B. and Smart, W.D. (2015) *Programming robots with ROS: A practical introduction to the Robot Operating System*. Sebastopol, CA: O'Reilly Media

Recalde, J.M., Dagnino, P., Mender, B.M. and Paz, R. (2016) 'Ventriloquism aftereffect in human subjects', *Journal of Neurophysiology*, 116(5), pp. 2340-2353

Rohde, M., van Dam, L.C.J. and Ernst, M.O. (2016) 'Statistically optimal multisensory cue integration: a practical tutorial', *Multisensory Research*, 29(4-5), pp. 279-317

ROS 2 Documentation (2022) *Humble Hawksbill release*. Available at: <https://docs.ros.org/en/humble/Releases/Release-Humble-Hawksbill.html>

ROS 2 Navigation2 Documentation (2024) *Navigation2 documentation*

ROS Index (2025) *nav2_regulated_pure_pursuit_controller*. Available at: https://index.ros.org/p/nav2_regulated_pure_pursuit_controller/

ROS Index (2026) *nav2_smac_planner*. Available at: https://index.ros.org/p/nav2_smac_planner/

Rösmann, C., Feiten, W., Wösch, T., Hoffmann, F. and Bertram, T. (2012) 'Trajectory modification considering dynamic constraints of autonomous robots', in *7th German Conference on Robotics (ROBOTIK 2012)*. Munich, Germany, pp. 1-6

Shams, L., Kamitani, Y. and Shimojo, S. (2000) 'What you see is what you hear', *Nature*, 408(6814), pp. 788

Siegwart, R., Nourbakhsh, I.R. and Scaramuzza, D. (2011) *Introduction to autonomous mobile robots*. 2nd edn. Cambridge, MA: MIT Press

Thrun, S., Burgard, W. and Fox, D. (2005) *Probabilistic robotics*. Cambridge, MA: MIT Press

van Beers, R.J., Sittig, A.C. and van der Gon, J.J.D. (1999) 'Integration of proprioceptive and visual position-information: An experimentally supported model', *Journal of Neurophysiology*, 81(3), pp. 1355-1364

Williams, G., Wagener, N., Goldfain, B., Drews, P., Rehg, J.M. and Theodorou, E.A. (2017) 'Information theoretic MPC for model-based reinforcement learning', in *2017 IEEE International Conference on Robotics and Automation (ICRA)*. Singapore, pp. 1714-1721